

Linux & Git Cheat Sheet			
1. Basic Linux Commands			
Command	Sample Usage	Description	Useful Flags / Notes
ls	ls -l	List files in directory	-a (show hidden), -h (human-readable sizes), -R (recursive)
cd	cd Documents	Change directory	cd .. = up one level; cd ~ = home; cd - = previous dir
pwd	pwd	Show current directory path	Useful for confirming your location before running scripts
mkdir	mkdir project	Create new directory	-p (create nested folders in one go, e.g. mkdir -p a/b/c)
rm	rm file.txt	Remove file	-r (recursive/folders), -f (force, no confirm) — irreversible
rmdir	rmdir old_folder	Remove empty directory	Fails if folder has contents; use rm -r instead
cp	cp file.txt backup.txt	Copy file	-r (copy folders recursively), -i (prompt before overwrite)
mv	mv file.txt folder/	Move/rename file	-i (prompt before overwrite); also used to rename in place
cat	cat notes.txt	Display file contents	Combine files: cat a.txt b.txt > c.txt
touch	touch newfile.txt	Create empty file	Also updates a file's last-modified timestamp
grep	grep "text" file.txt	Search text inside a file	-i (ignore case), -r (recursive search in folders), -n (line numbers)
find	find . -name "*.txt"	Search for files by name	-type f (files only), -type d (dirs only), -mtime (by date)
chmod	chmod +x script.sh	Change file permissions	See permission table below (section 6)
chown	chown user:group file.txt	Change file owner/group	Often needs sudo to run
head / tail	tail -f log.txt	View start/end of a file	-n 20 (show 20 lines); -f (live-follow a growing file, e.g. logs)
ps / kill	kill -9 1234	List/stop running processes	ps aux lists all; -9 forces termination
df / du	df -h	Show disk/folder space usage	-h = human-readable (MB/GB); du -sh * = size per item
man	man ls	Show manual/help for a command	Press q to quit the manual viewer

2. Basic Git Commands

Command	Sample Usage	Description	Common Options	Notes / Best Practice
git init	git init	Start new repo	—	Run once, in your project's root folder
git status	git status	Show changes & staging	-s (short)	Run often — it's the safest command in Git
git add	git add file.txt	Stage changes	. (all files)	Review with git status before committing
git commit	git commit -m "msg"	Save snapshot	-a (all tracked)	Write clear, present-tense messages (e.g. "Fix login bug")
git log	git log --oneline	Show commit history	--graph	Add --all to see history across every branch
git diff	git diff file.txt	Compare changes	--staged	Plain git diff = unstaged; --staged = staged vs last commit
git restore	git restore file.txt	Undo changes in file	--staged	Replaces older git checkout -- file usage
git branch	git branch feature	Create/manage branches	-d (delete)	Keep branch names short and descriptive
git checkout	git checkout feature	Switch branch	-b (new branch)	Commit or stash changes first to avoid losing work
git switch	git switch feature	Easier branch switching	-c (create)	Newer, safer alternative to git checkout for branches
git merge	git merge feature	Combine branch changes	--no-ff	Resolve conflicts, then git add + git commit
git tag	git tag v1.0	Mark commits	-a (annotated)	Commonly used to mark release versions
git clone	git clone URL	Copy repo	--depth=1 (shallow)	Downloads full history unless --depth is used
git pull	git pull origin main	Fetch and merge remote changes	--rebase	Pull often to avoid large, messy merge conflicts
git push	git push origin main	Upload local commits to remote	-u (set upstream)	-u links your branch so future pushes just need git push
git stash	git stash	Temporarily save uncommitted changes	pop, list	Handy for quickly switching tasks without committing
git rebase	git rebase main	Reapply commits on top of another base	-i (interactive)	Keeps history linear; avoid rebasing shared/public branches
git remote	git remote -v	Show/manage remote repo links	add, remove	"origin" is the default name for your main remote

3. Git's Three-Tree Architecture

- **Working Directory** → Your actual files on disk (where you edit).
- **Staging Area (Index)** → Snapshot of changes you plan to commit.
- **Repository (HEAD)** → Permanent history of commits.

Working Directory → git add → Staging Area → git commit → Repository (HEAD)

4. File States in Git

- **Untracked** — new file Git doesn't know about yet.
- **Modified** — tracked file changed in the Working Directory.
- **Staged** — modified file added to the Staging Area via git add.
- **Committed** — staged file safely saved in the Repository via git commit.

5. Typical Beginner Git Workflow

- **1.** git init OR git clone URL — start or copy a repo.
- **2.** Edit files in your **Working Directory**.
- **3.** git status — check what changed.
- **4.** git add . — move changes to the **Staging Area**.
- **5.** git commit -m "message" — save the snapshot to the **Repository**.
- **6.** git push origin main — upload commits to the remote.
- **7.** git pull origin main — get the latest changes from teammates.
- **8.** Repeat: edit → add → commit → push.

6. Common File Permission Codes (chmod)

Code	Permission	Meaning	Example Command
755	rwxf-xf-x	Owner: read/write/execute; Group & Others: read/execute	chmod 755 script.sh
644	rw-r--r--	Owner: read/write; Group & Others: read only	chmod 644 notes.txt
700	rwX-----	Owner: full access; Group & Others: no access	chmod 700 private.sh

References

- Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. Retrieved from <https://git-scm.com/book/en/v2>
- Git Documentation. (n.d.). *Git reference manual*. Retrieved from <https://git-scm.com/docs>
- Atlassian. (n.d.). *Git tutorials and training*. Retrieved from <https://www.atlassian.com/git/tutorials>
- Torvalds, L. (2005). *Git source code management system*. Retrieved from <https://github.com/git/git>
- Ubuntu Documentation. (n.d.). *Basic Linux commands*. Retrieved from <https://help.ubuntu.com>

"The author used Claude.ai to organize and compact every collected info into a single A4 page during assignment preparation. The author reviewed and edited the content as needed and takes full responsibility for the final content of the work."